

Schema v1 — validação empírica em 2 modelos

Date: 2026-05-30

Smoke real do [STRUCTURED_OUTPUT=v1] em dois modelos contrastantes pra checar se a feature de saída padronizada funciona na prática.

Modelos testados

Modelo	Tamanho	Backend	Role
deepseek/deepseek-v4-flash	~37B (proprietary)	OpenRouter API	grande / cloud
Qwen/Qwen2.5-Coder-3B-Instruct	3B	transformers local CPU fp32	pequeno / coder

Por que esses dois: pra responder 'schema funciona só em modelo grande ou em qualquer coder?'. Gemma-4B (general-purpose) já tinha mostrado parse=0/16 em smoke anterior — esperávamos Qwen 3B se comportar parecido. Não foi o caso.

Resultados

Diagnostic	DeepSeek V4 Flash	Qwen 2.5 Coder 3B
Backend	OpenRouter API	transformers local CPU
N invocations	4 (cycle 1 OK)	4 sequenciais
parse_ok / N	3 / 4 (75%)	4 / 4 (100%)
parse_failed	1	0
behavior groups	4	4
modal_count	1	1
winner.confidence	1.0	0.9
expected_oracle_pass	2 expectativas	5 expectativas (assert-shaped)
artifact correto?	✓	✓
Wall-clock total	16.4s (4 paralelos)	~470s (4 sequenciais CPU)

Achado-chave #1: schema funciona em modelo 3B coder

Expectativa antes do teste: small models ($\leq 7B$) parse failure rate alto, baseada no Gemma-4B (parse=0/16). Qwen 2.5 Coder 3B refutou isso com 4/4 (100%) parse_ok.

A diferença não é tamanho — é especialização:

Modelo	Tamanho	Tipo	parse_ok rate
Gemma-4B-it	4B	general	0 / 16 (0%)
DeepSeek V4 Flash	~37B	general (chat)	3 / 4 (75%)
Qwen 2.5 Coder 3B	3B	coder-specialized	4 / 4 (100%)

Hipótese: coder-tuned models seguem instructions estruturadas melhor que general models do mesmo tamanho. Treinados em MUITO JSON/YAML/schemas, então 'retorne JSON com esses 6 campos' cai num padrão familiar.

Implicação: schema v1 não é feature apenas pra modelo grande. Coder models, mesmo pequenos (3B), participam.

Achado #2: Qwen 3B preenche expectativas com substância real

A `winner.expected_oracle_pass` do Qwen 3B veio com 5 asserts concretos:

```
assert(PasswordPolicy::isStrong('a') === false);
assert(PasswordPolicy::isStrong('abc') === false);
assert(PasswordPolicy::isStrong('abcdefg') === false);
assert(PasswordPolicy::isStrong('abcdefg123') === true);
assert(PasswordPolicy::isStrong('12345678901234567890') === true);
```

Não é placeholder. **O modelo articulou expectativas testáveis sobre o próprio output.** Isso é exatamente o que o schema foi desenhado pra capturar — sinal pra detecção de divergência parse-vs-oracle.

Achado #3: escalator parou em cycle 1 nos 2 modelos

`cycles_run=1` em ambos. Oracle (heurística text-only no smoke) passou na primeira rodada. Confirma que pra tasks fáceis o gradual escalation economiza calls — não passa pro tier 2.

Achado #4: custo prático

Modelo	Calls	Tempo	\$\$
DeepSeek V4 Flash	4	16 s	< \$0.01 OR
Qwen 3B local	4	8 min	\$0 (CPU) + ~6GB RAM

CPU é viável pra batch pequeno; pra batch real (>100 calls) precisa GPU.

Limitações do smoke

1. **N=4 é pequeno** pra modal-vote real (precisa N=64+ pra consenso emergir). Validamos só a honrança do schema, não a vitória do behavior-modal-vote sobre raw-string-modal.
2. **Oracle do smoke é heurístico** (texto contém 'isStrong' e '12'), não real PHPUnit. Pra validação completa, precisa rodar bench v13 com schema ON.
3. **1 task só** (password_strength). Pra extrapolar pra todos os 12 cases, precisa do batch completo.
4. **Não testamos modelo medium** (Gemma-4B). O 0/16 vem do bench anterior; re-teste com schema atual seria útil.

O que isso desbloqueia

Antes desse smoke: hipótese era schema v1 = luxo de modelo grande, modelos $\leq 7B$ caem pro fallback raw. **Refutado.**

Implicações práticas:

- **simplicio task pode adotar schema v1 mesmo com doer pequeno** (Qwen 2.5 Coder 3B, Coder 7B).

Modal-vote-por-behavior melhora pra qualquer coder.

- **sp_fanout_escalating(structured=True) pode ser default** pra coder models, com fallback automático nos outros (general models pequenos).
- **Issue de adoção upstream no simplicio-prompt** ganha argumento empírico: 'funciona em modelo 3B coder, é prática viável'.

Próximo passo recomendado

Re-rodar o bench v13 completo (3 modelos $\times$ 12 cases $\times$ 5 lados) com schema v1 ATIVO no lado sp. Comparar cli+sp v13 antigo (sem schema, string modal) vs cli+sp v14 (com schema, behavior modal).

Custo: ~3000 chamadas, ~\$3, ~1h. Hipótese: schema melhora pass-rate dos lados sp em modelos coder porque modal-vote-por-behavior agrega melhor.